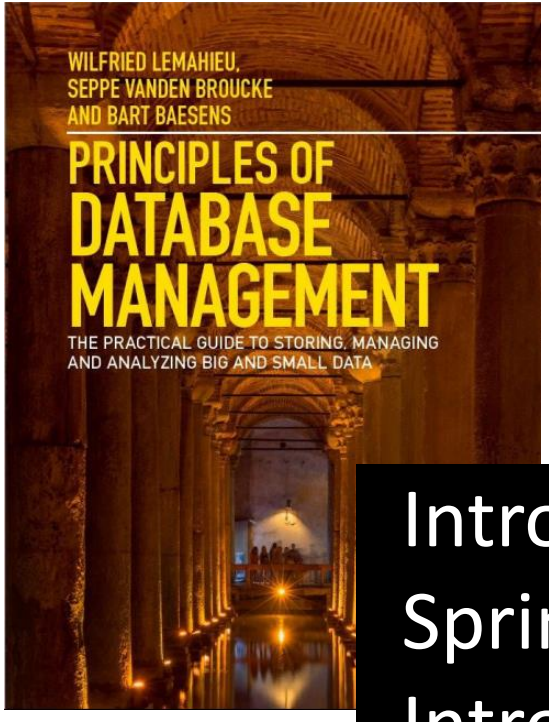


Introduction to Database Systems
Spring 2025, Lecture 1:
Introduction & Relational Model

Agenda

Introduction

- Lecturers & Teaching Assistants
- Intended Learning Outcomes
- Course Overview
- DuckDB

Relational Model

Lecturer & Course Responsible

Martin Hentschel

IT UNIVERSITY OF COPENHAGEN — Associate Professor

snowflake — Principal Software Engineer

Microsoft — Software Engineer

ETH zürich — BSc, MSc, PhD in Computer Science

Lecturer

Omar Khan

Postdoctoral Researcher
incl. Lecturer at ITU

IT UNIVERSITY OF COPENHAGEN

Postdoctoral Researcher

IT UNIVERSITY OF COPENHAGEN

BSc, MSc, PhD in Computer Science

Teaching Assistants

Anne-Marie Rommerdahl

Christina Aftzidis

Ahmet Talha Akgül

Anders Arvesen

Andreas Vinkler

Philip Flyvholm

Research Group

DASYA

https://dasya.itu.dk/

Data-Intensive Systems and Applications

17 members: professors, postdoc researchers, PhD students

Focusing on complete data lifecycle

- Data collection
- Data storage
- Data processing incl. ML

Research projects available on website

Phones out!

"Database systems"– what do you expect to do with it?

www.menti.com Code: 5704 8726

Agenda

Introduction

- Lecturers & Teaching Assistants
- Intended Learning Outcomes
- Course Overview
- DuckDB

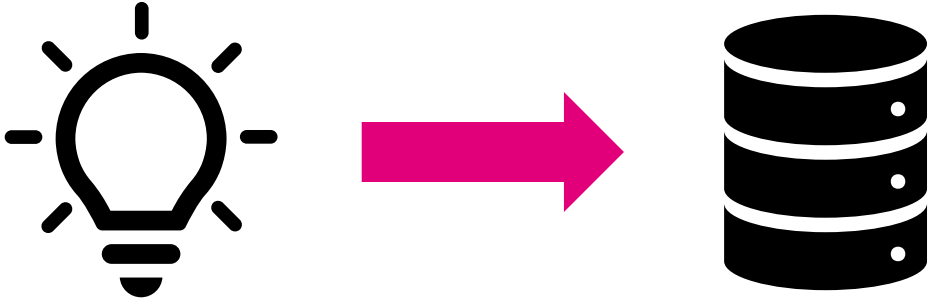
Relational Model

Intended Learning Outcomes

1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.
3. Use relational DBMSs from a conventional programming language in a secure manner.
4. Analyze/predict/improve query processing efficiency of the designed database using indices.
5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design.
6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

ER Modeling

1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.

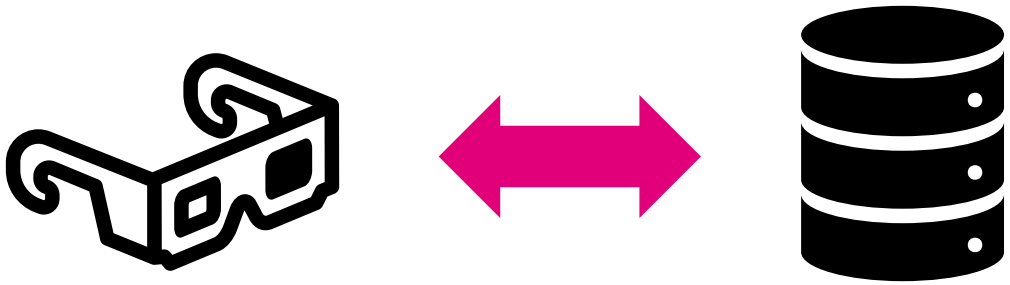


Convert your idea of an app into a database schema

game
web site
business application
anything that stores data

SQL Language

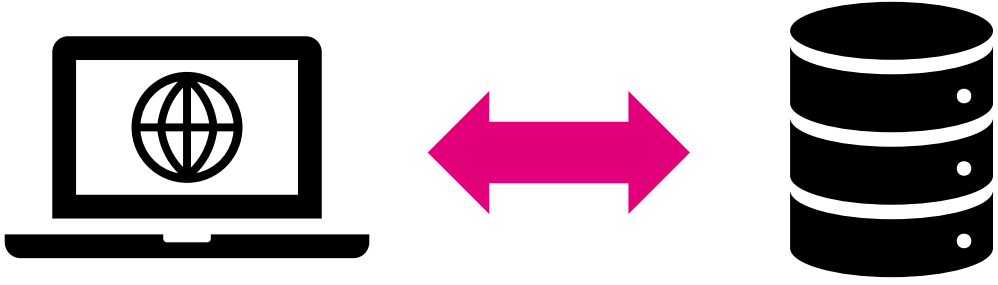
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.



Work with the data stored in the database

Python and more

3. Use relational DBMSs from a conventional programming language in a secure manner.



Automate everything

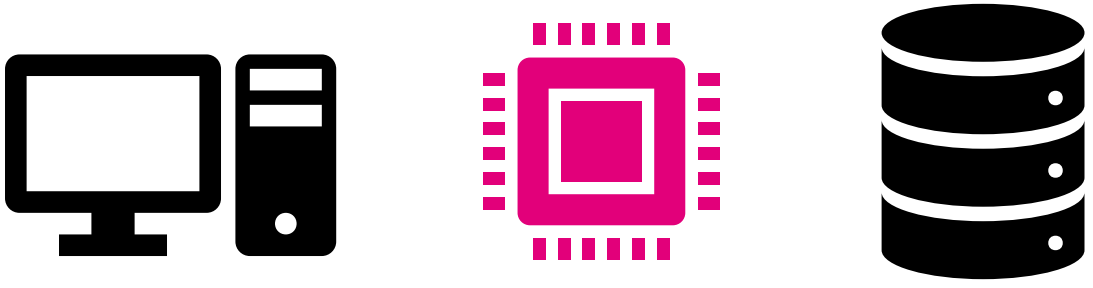
Performance

4. Analyze/predict/improve query processing efficiency of the designed database using indices.

Make it fast

Hardware and Storage

5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design



Understanding the internals to make better decision

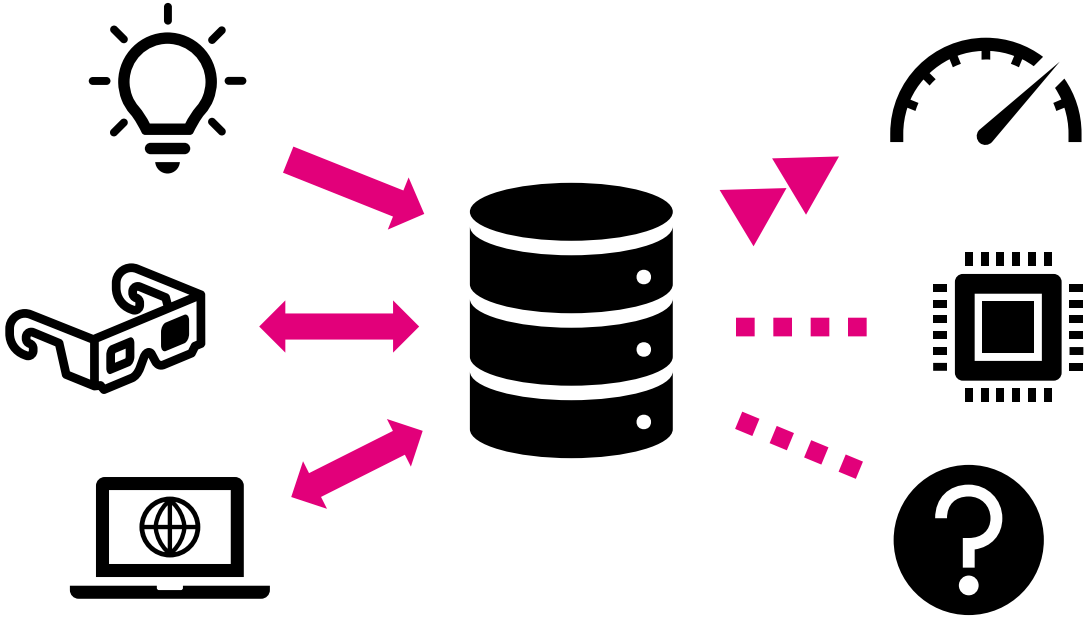
Data Management Systems

6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

"The right tool for the right job"

Putting it Together

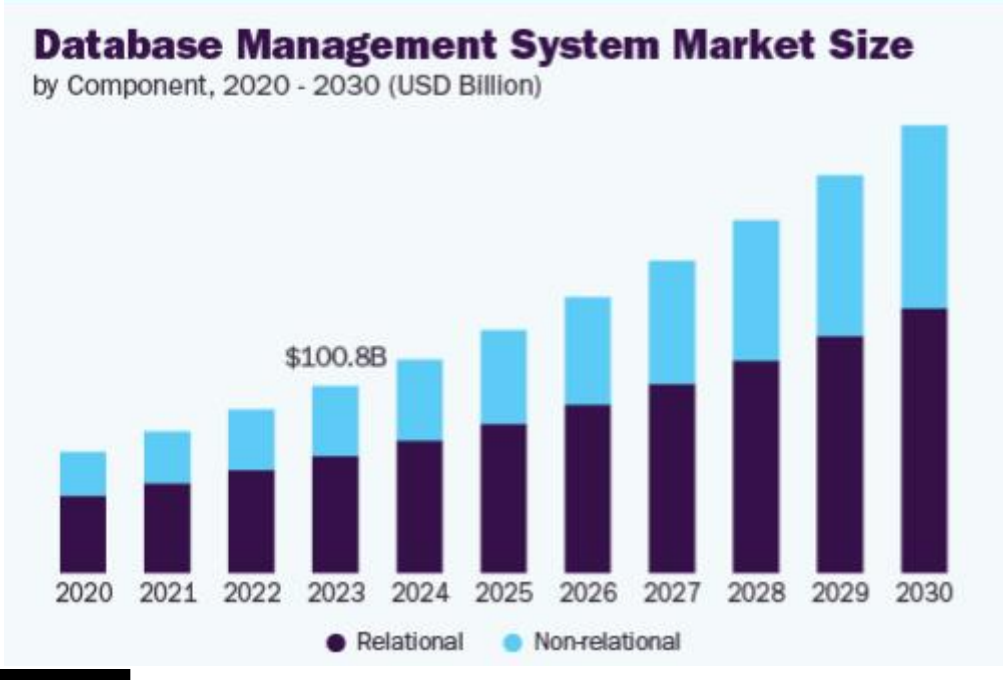
You will learn to put an idea* into practice, work with it, scale it, and understand the tech behind it.



*provided the idea is related to data

Why are Databases Important?

- Crucial to effectively manage data of all sizes
- Ease application development
- Maintain data quality, security, governance



Source: Database Management System Market Size Report, 2030

Agenda

Introduction

- Lecturers & Teaching Assistants
- Intended Learning Outcomes
- Course Overview
- DuckDB

Relational Model

Course Schedule

Date	Topics	Teacher
January 29	Introduction, Relational model	Martin
February 5	ER modeling, SQL DDLs	Omar
February 12	Normal forms	Martin
February 19	SQL querying 1	Martin
February 26	SQL querying 2	Martin
March 5	Programming language support	Omar
March 12	RBAC, Database internals	Martin
March 19	Join algorithms, Min/max indexes	Martin
March 26	B-tree indexes, JSON	Omar
April 2	ACID, Transactions	Omar
April 9	OLTP/OLAP, Data warehouses	Martin
April 16	– EASTER BREAK –	
April 23	Big Data Processing, Data lakes	Martin
April 30	Test exam	
May 7	– NO CLASS –	

Course Structure

Lectures

- Wednesdays, 10:15—12:00
- Room: AUD1

Exercises

- Wednesdays, 12:15—14:00
- Rooms: 2A52, 2A54, 3A12—14
- Room 2A52 will be used for in-depth walkthroughs through exercises

Homework

- 4 homework assignments
- Mandatory 3 out of 4, 20% passing grade

Discussion Forum

- Piazza: https://piazza.com/itu.dk/spring2025/idbss25

Homework Schedule

Homework	Hand-out	Hand-in	Topics
HW1	February 5	February 19 at 23:59	ER, DDL, inserts
HW2	February 19	March 5 at 23:59	NF, SQL
HW3	March 12	March 26 at 23:59	Python + prepared statements, SQL window functions, RBAC
HW4	March 26	April 9 at 23:59	Indexes, Transactions, JSON

Exercises Schedule

Date	Exercise	Topics
January 29	EX1	Install and use DuckDB
February 5	EX2	ER models to DDLs
February 12	EX3	Normal forms
February 19		Work on HW1
February 26	EX4	SQL
March 5		Work on HW2
March 12	EX5	Advanced SQL (incl. windows), Python
March 19	EX6	Indexes, join algorithms
March 26		Work on HW3
April 2	EX7	ACID, transactions
April 9		Work on HW4
April 16		– EASTER BREAK –
April 23	EX8	Parquet, JSON, OLTP/OLAP
April 30		Test Exam
May 7		No exercise assignment, TAs are available for questions

Homework during Exercise Sessions

Date	Exercise	Topics
January 29	EX1	Install and use DuckDB
February 5	EX2	ER models to DDLs
February 12	EX3	Normal forms
February 19		Work on HW1
February 26	EX4	SQL
March 5		Work on HW2
March 12	EX5	Advanced SQL (incl. windows), Python
March 19	EX6	Indexes, join algorithms
March 26		Work on HW3
April 2	EX7	ACID, transactions
April 9		Work on HW4
April 16		– EASTER BREAK –
April 23	EX8	Parquet, JSON, OLTP/OLAP
April 30		Test Exam
May 7		No exercise assignment, TAs are available for questions

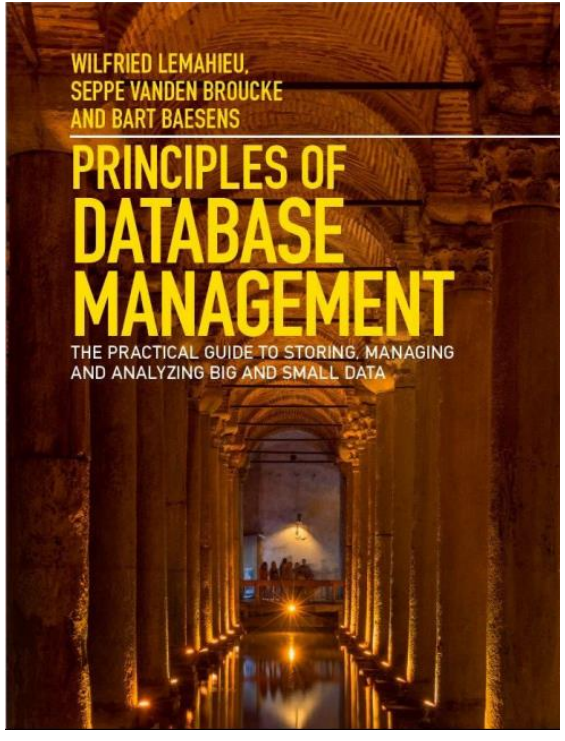
Agenda

Introduction

- Lecturers & Teaching Assistants
- Intended Learning Outcomes
- Course Overview
- DuckDB

Relational Model

Book and Database System



DuckDB

Why DuckDB

DuckDB

Very popular, new database system

- **Easy to install and use**
- Runs on Windows, Mac, Linux
- Rich SQL including support for CSV, Parquet, JSON
- Free and open-source

Started by the CWI research group in Amsterdam

DuckDB Installation

DuckDB

Installation instructions: https://duckdb.org/docs/installation/

Download and unzip the executable (Linux)

```
~$ wget https://github.com/duckdb/duckdb/releases/
download/v1.1.3/duckdb_cli-linux-amd64.zip
~$ unzip duckdb_cli-linux-amd64.zip
Archive:  duckdb_cli-linux-amd64.zip
  inflating: duckdb
```

Windows/Mac versions available as well. Download via PowerShell/Terminal or via the browser

DuckDB Usage

DuckDB

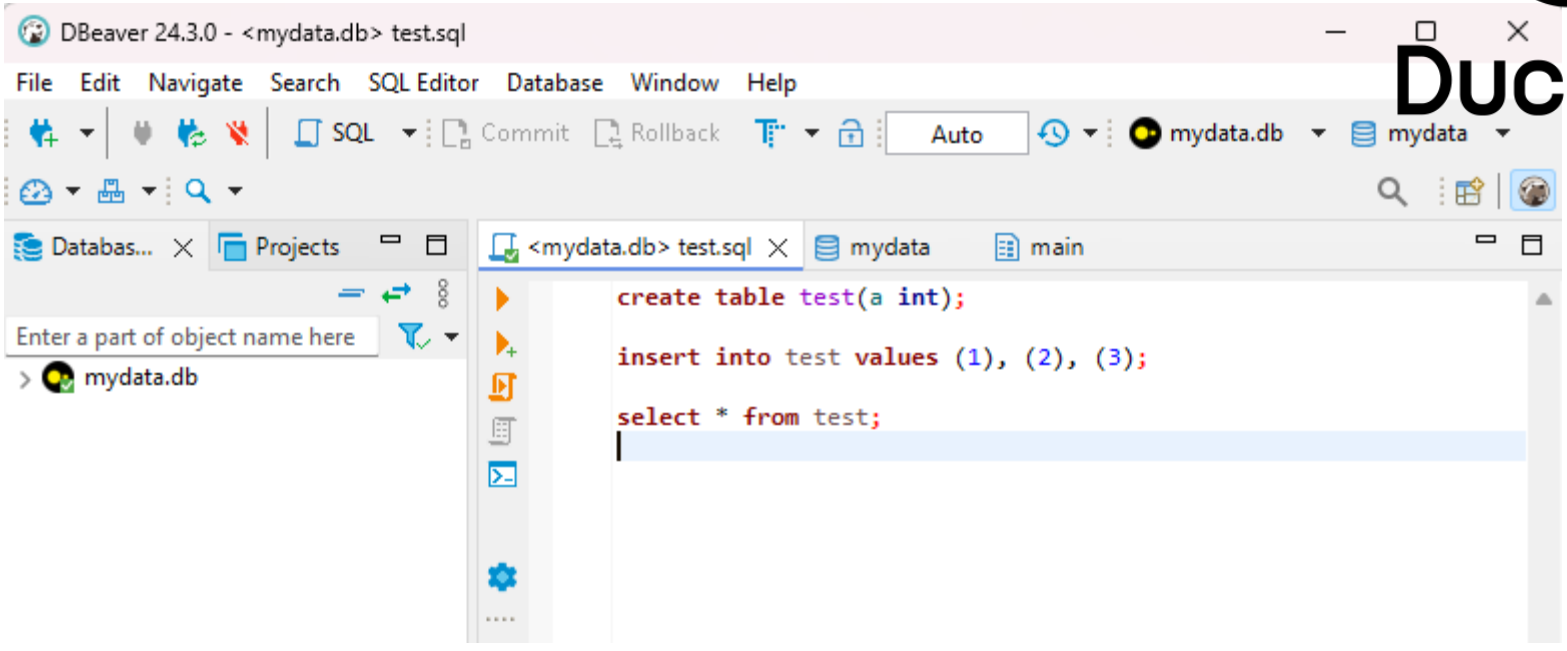
Run DuckDB (Linux)

```
~$ ./duckdb mydata.db
v1.1.3 19864453f7
Enter ".help" for usage hints.
D create table test(a int);
D insert into test values (1), (2), (3);
D select * from test;
┌─────────┐
│    a    │
│  int32  │
├─────────┤
│       1 │
│       2 │
│       3 │
└─────────┘
```

That's where the data is stored. If the file does not exist, DuckDB creates it.

Exit via typing ".exit" or CTRL-d on Linux, CTRL-c on Windows

DBeaver Client UI



If you want, install [DBeaver SQL IDE]

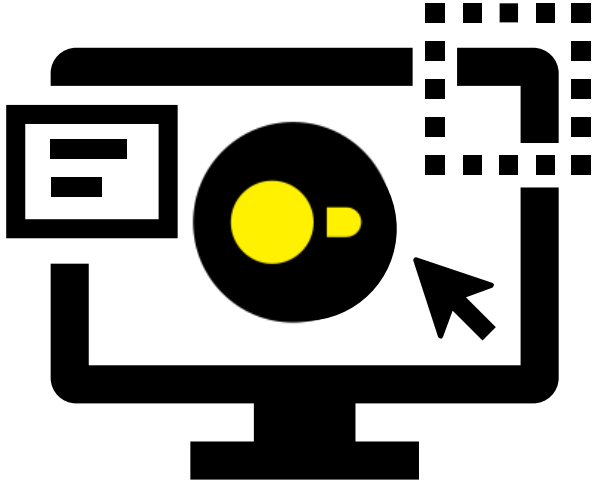
- Keep track of all your SQL queries
- **Tip**: Execute the query at the cursor with **CTRL+ENTER**

DuckDB Installation

DuckDB

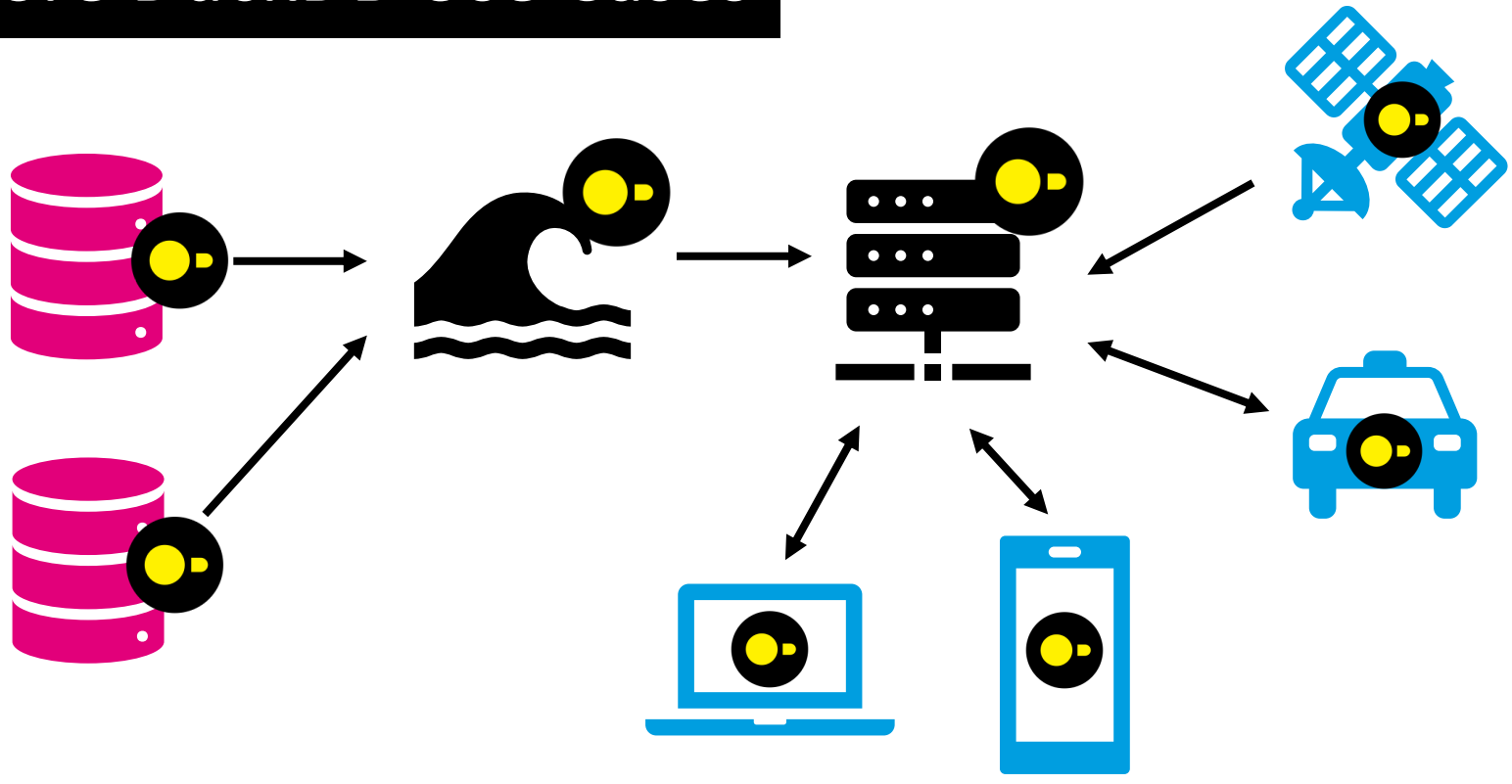
Demo

DuckDB Use Case



Data analysis of data that fits on a single machine

More DuckDB Use Cases

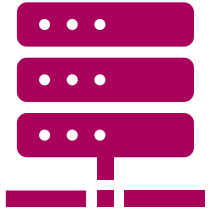


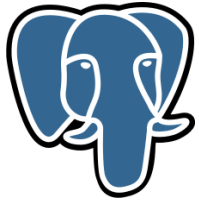
Data extraction, data lake processing, processing of data on the “edge” (NASA, Tesla, …)

Other Important Database Use Case

Three-Layer Architecture:

1. Front-end: websites, apps
2. Middleware: web server
3. Database

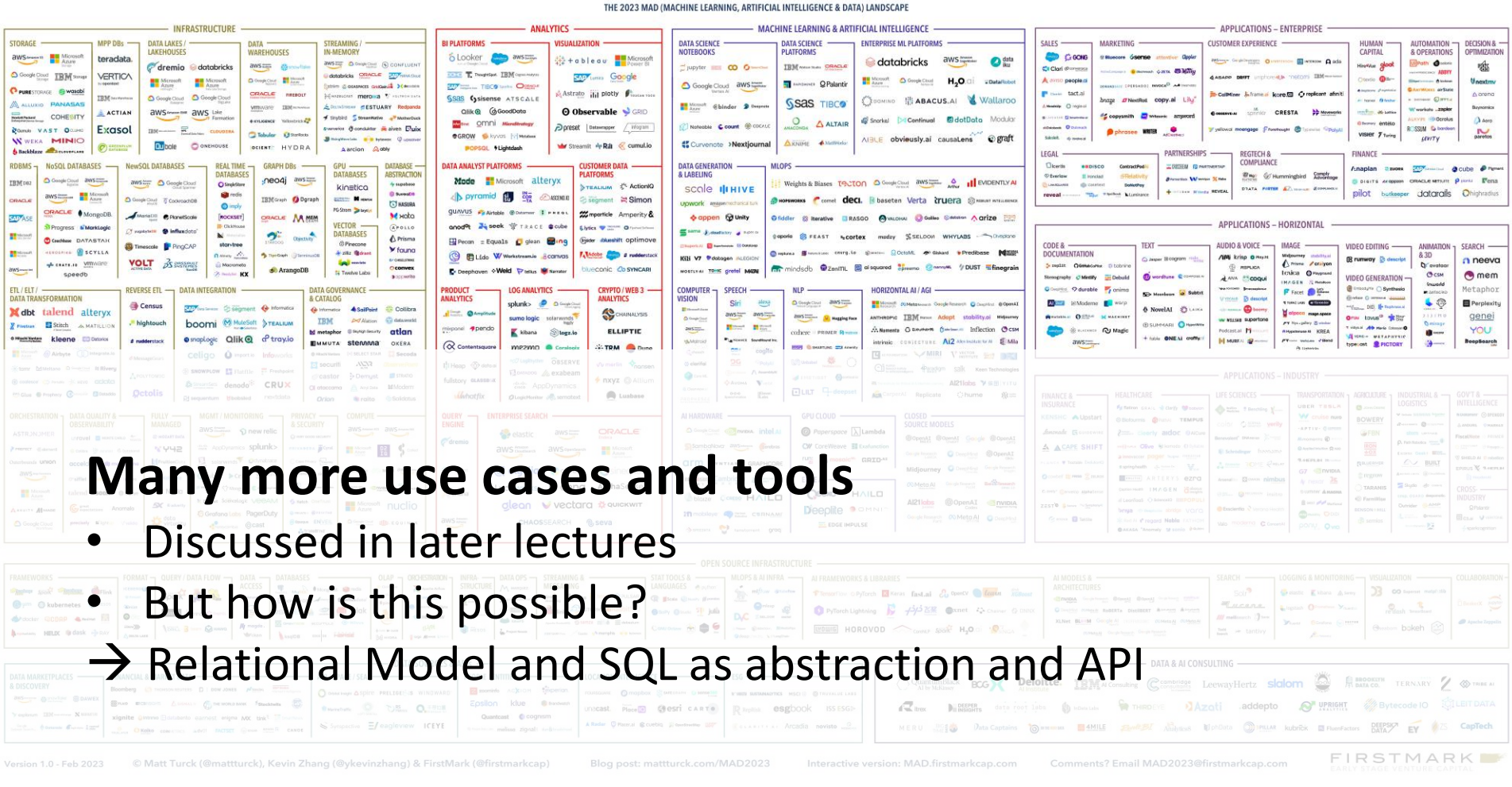




Example: LAMP stack

- Linux
- Apache HTTP server
- MySQL or Postgres database
- PHP / Python / Perl

Database Use Cases



Many more use cases and tools

- Discussed in later lectures
- But how is this possible?

→ Relational Model and SQL as abstraction and API

Source: MAD 2023 - Version 1.0

Agenda

Introduction

- Lecturers & Teaching Assistants
- Intended Learning Outcomes
- Course Overview
- DuckDB

Relational Model

Relational Database Definition

A relational database is a type of database that is based on the relational model

- stores data in a set of tables with rows and columns (a.k.a. relations)
- uses relationships between these tables to manage the data

A relational database system (RDBMS) implements and manages relational databases

Pros:

- Data independence
- Abstract representation allows for many optimizations

→ RDBMS dominate the databases market*

*Michael Stonebraker, Andrew Pavlo, What Goes Around Comes Around… And Around…, SIGMOD Record, June 2024
https://db.cs.cmu.edu/papers/2024/whatgoesaround-sigmodrec2024.pdf

Relational Model

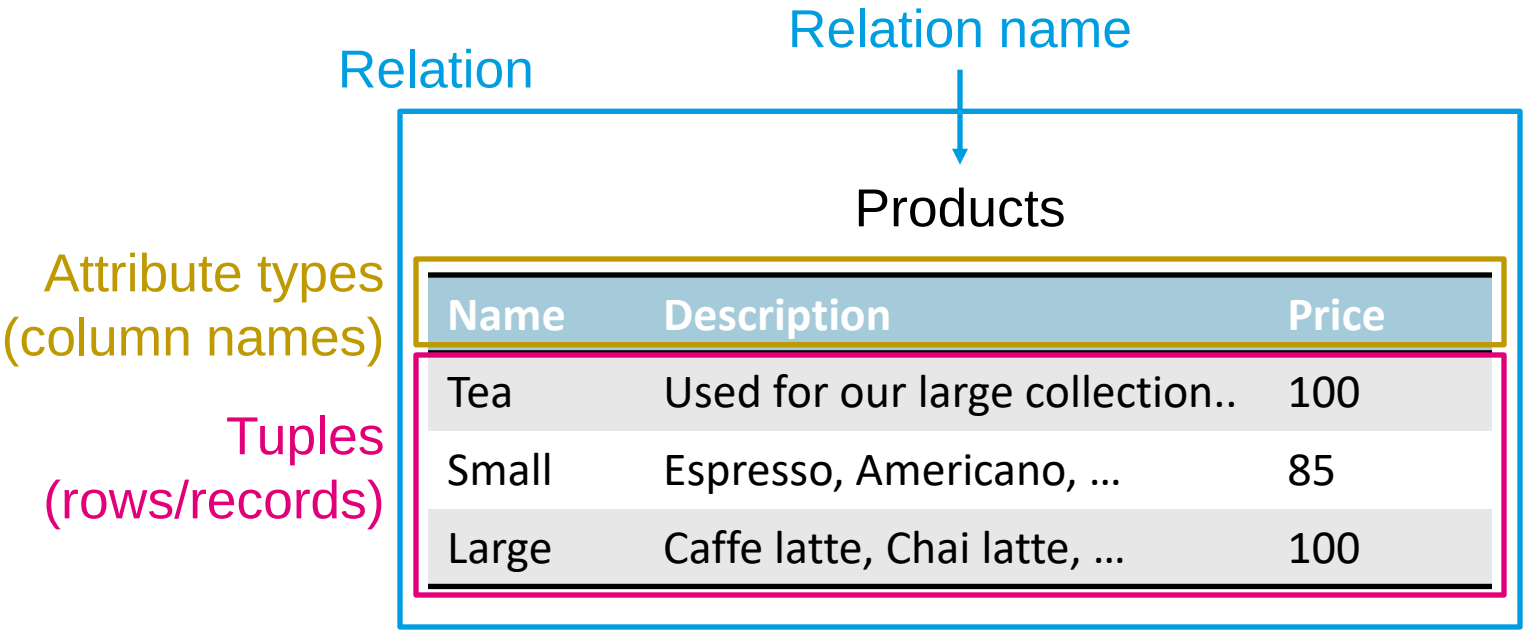
See Book, Chapter 6

Relation: a set of *tuples* that each represent a real-world entity (e.g. user, product, purchase)

Tuple: ordered list of attribute *values* that each describe an aspect of the entity (e.g. product name, price)

→ A relation can be visualized as a table of values

Relational Model



Written as

Products(Name, Description, Price)

Schema vs. Instance vs. Database

Schema: Relation name + list of attributes

- Optional: attribute types
  Products(Name: string, Description: string, Price: int)

Instance: Tuples in a relation

Database: Collection of relations (i.e. tables)

Example of a Database

Products (Name: string, Description: string, Price: int)

Users (Name: string, Email: string, Password: string, Salt: string, CreatedOn: date)

Purchases (ProductName: string, UserEmail: string, CreatedOn: date)

Keys and Superkeys

What is a key?

- Defines unique records (tuples)
- Helps in setting relationships between relations
- Ensures the mathematical definition of a relation (set of tuples)

Superkey

- Set of attributes that uniquely identifies records: uniqueness property
- Entire set of attributes of a relation is a superkey
- Minimal superkey: Minimality property = no attributes can be removed from a superkey without violating the uniqueness property

Candidate Keys and Primary Keys

Candidate Keys

- Attributes that satisfy the uniqueness and minimality properties
- Minimal superkey = candidate key
- Relation can have many candidate keys

Primary Key

- One candidate key gets designated as primary key
- Used to identify tuples in the relation

2-Minute Exercise

What are superkeys and candidate keys? What is the best key for being the primary key? What key does not make sense in practice?

Products (Name: string, Description: string, Price: int)

Name	Description	Price
Tea	Used for our large collection..	100
Small	Espresso, Americano, …	85
Large	Caffe latte, Chai latte, …	100

Options:
(Name)
(Description)
(Price)
(Name, Description)
(Name, Price)
(Description, Price)
(Name, Description, Price)

2-Minute Exercise

What are superkeys and candidate keys? What is the best key for being the primary key? What key does not make sense in practice?

Products (Name: string, Description: string, Price: int)

Name	Description	Price
Tea	Used for our large collection..	100
Small	Espresso, Americano, …	85
Large	Caffe latte, Chai latte, …	100

Options:

(Name) — candidate key (best key)
(Description) — candidate key (does not make sense in practice)
(Price) — none
(Name, Description) — superkey
(Name, Price) — superkey
(Description, Price) — superkey
(Name, Description, Price) — superkey

Another 2-Minute Exercise

What are superkeys and candidate keys? What is the best key for being the primary key? What key does not make sense in practice?

Users (Name: string, Email: string, Password: string, Salt: string, CreatedOn: date)

Purchases (ProductName: string, UserEmail: string, CreatedOn: date)

Another 2-Minute Exercise

What are **candidate keys**? What is the best key for being the primary key? What key does not make sense in practice?

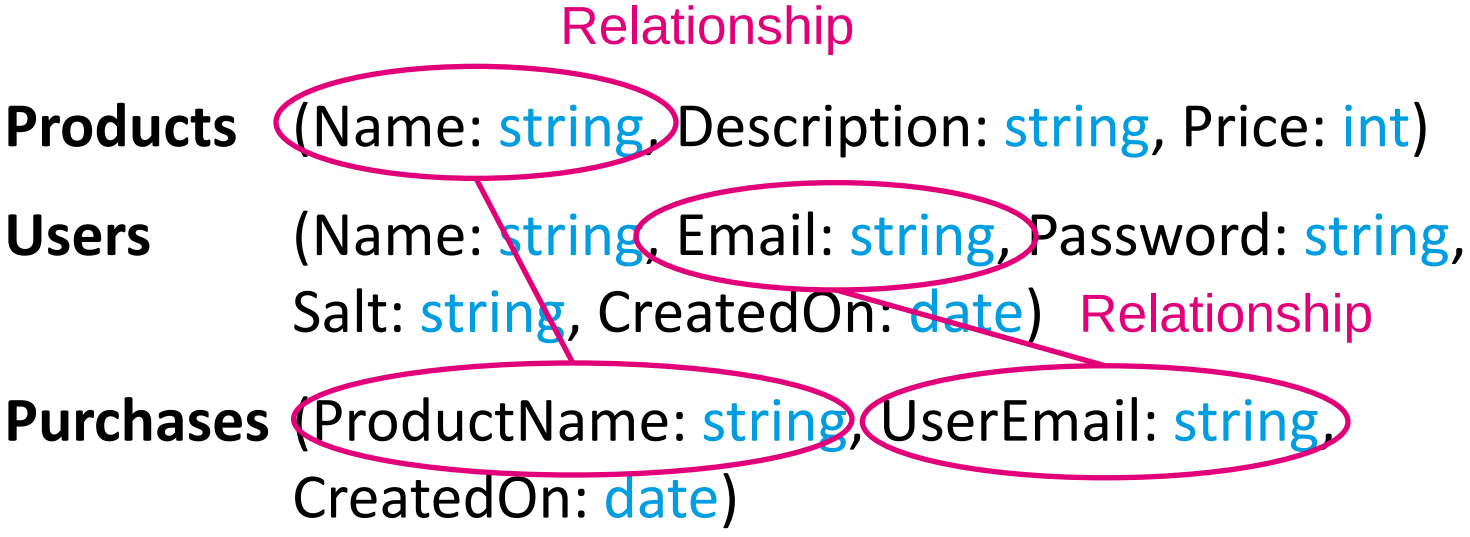
Users (Name: string, Email: string, Password: string, Salt: string, CreatedOn: date)

Candidate keys: (Name) if application enforces unique usernames
(Email) if user must enter email address upon creation
No keys: (Password), (Salt), (CreatedOn) or any combination of these

Purchases (ProductName: string, UserEmail: string, CreatedOn: int)

Candidate keys: (ProductName, UserEmail, CreatedOn)
No keys: (ProductName), (UserEmail), (CreatedOn), (ProductName, UserEmail)

Relationships between Relations



Foreign Keys

Defines the relationship between relations

Definition: A set of attribute types FK in a relation R is a foreign key if:

1. Attribute types in FK match a primary key PK in relation S
2. Any record in R has a value in FK that occurs as a value of PK in S or is NULL

→ Called “Referential integrity constraint”

Foreign Keys: Example

Products

PK

Name	Description	Price
Tea	Used for our large collection..	100
Small	Espresso, Americano, …	85
Large	Caffe latte, Chai latte, …	100

Users PK

Name	Email	Password	…
Anna	anna@itu.dk	*%x	…
Martin	mhent@itu.dk	§$%	…
Tim	tim@itu.dk	=)/(	…

FK

Purchases FK

Product Name	UserEmail	CreatedOn
Tea	anna@itu.dk	Jan 27, 10:05
Small	mhent@itu.dk	Jan 27, 11:04
Small	mhent@itu.dk	Jan 27, 14:12

Takeaways

Welcome to the new semester!

Databases enable you to build great applications!

Relational model

- Logical data model representing structured data
- Defines relations and tuples
- Candidate keys, primary keys identify tuples
- Foreign keys represent relationships between relations